

Section 5.2: Cache Synchronization, Update Strategies & Failure States

Study Guide | Part 2 of 4 | System Design Interview Prep Series

Executive Overview

Choosing the wrong cache update strategy is a common source of subtle production bugs — stale data, doubled DB load, or lost writes. This section systematically covers the five canonical cache update patterns, three critical vulnerability vectors (penetration, avalanche, stampede), and how to reason about each in interviews.

Core mental model: The update strategy determines *when* and *how* cache and DB stay in sync. The invalidation strategy determines *what* to evict and *when*. They're independent decisions — choose each based on your consistency and latency requirements.

Cache Update Topologies

1. Cache-Aside (Lazy Loading)

The application is responsible for reading from and writing to both cache and DB. The cache is populated only on cache miss (lazy).

Read path:

App → Cache.get(key) → miss → DB.query() → Cache.set(key, val, TTL) → return val

Write path:

App → DB.write(data) → Cache.delete(key) → return

```
def get_user(user_id: int):
    raw = redis.get(f"user:{user_id}")
    if raw:
        return deserialize(raw)

    user = db.query("SELECT * FROM users WHERE id = ?", user_id)
```

```

if user:
    redis.setex(f"user:{user_id}", serialize(user), ttl=3600)
else:
    # Cache null results with short TTL to prevent penetration
    redis.setex(f"user:{user_id}:null", "1", ttl=60)
return user

def update_user(user_id: int, data: dict):
    db.execute("UPDATE users SET ... WHERE id = ?", user_id)
    redis.delete(f"user:{user_id}") # Invalidate, not update

```

Why delete instead of update on write? Updating the cache on write creates a race condition:

```

Thread A: DB write → Thread B: DB write → Thread B: Cache set(v2) → Thread A:
Cache set(v1)
Result: Cache has v1 (stale), DB has v2. Incorrect.

Deleting avoids this: next read fetches v2 from DB and re-populates correctly.

```

Best fit: General-purpose default for most services. Cache failures don't break reads (just slower). Works naturally with connection pooling and horizontal scaling.

Watch out for: Cold start — first traffic burst after deployment has 100% cache misses.

2. Read-Through

The cache itself is responsible for fetching from the DB on a miss. The application only talks to the cache.

```
App → Cache.get(key) → miss → Cache loads from DB automatically → return val
```

Implementation: Provided by cache libraries (e.g., Spring Cache, Caffeine, NCache) via a `CacheLoader` interface.

```

LoadingCache<String, User> cache = Caffeine.newBuilder()
    .maximumSize(100_000)
    .expireAfterWrite(1, HOURS)
    .build(key -> db.loadUser(key)); // CacheLoader – called on miss

// Application only calls cache.get()
User user = cache.get("user:123"); // Transparent DB fetch if miss

```

Best fit: When you want consistent caching behavior enforced at the infrastructure level, not scattered across application code.

Trade-off: Cache becomes a hard dependency. If the cache cluster is down, reads fail (unlike cache-aside where you can fall through to DB). Adds vendor coupling.

3. Write-Through

Every write goes to the cache first, which synchronously writes through to the DB. Both are always in sync.

```
App → Cache.set(key, val) → [Cache writes to DB] → return (both updated)
```

Best fit: Workloads that read back data immediately after writing. Avoids the post-write cache miss. Works well for user session data, shopping carts.

Trade-off: Every single write incurs both a cache write and a DB write latency. If you write 1000 records that are never read, you've wasted 1000 cache writes. Write amplification hurts write-heavy workloads.

4. Write-Behind (Write-Back)

Writes go to the cache immediately; a background process batches and flushes to DB asynchronously.

```
App → Cache.set(key, val) → return immediately (fast!)
Background: queue → batch DB writes every N ms
```

```
# Write to cache immediately
redis.set(f"counter:{item_id}", new_value)
write_queue.append((item_id, new_value, time.time()))

# Background flusher (runs every 100ms)
def flush_write_queue():
    batch = deque_batch(write_queue, max_size=500)
    db.bulk_update(batch)
```

Best fit: High-throughput write scenarios where DB write latency is the bottleneck — view counters, like counts, analytics events, shopping cart updates.

Critical risk: If the cache node crashes before the background flush, writes are lost. Must have persistence (Redis AOF) or a durable queue (Kafka) as a buffer.

5. Refresh-Ahead

The cache proactively refreshes hot keys *before* they expire, preventing miss latency.

```
TTL = 300s
At 250s (83% through TTL) → background refresh triggered
User at 260s → still gets cached value, no miss penalty
```

```
def get_with_refresh(key: str):
    val, meta = redis.get_with_ttl(key)
    remaining_ttl = meta['ttl']
    original_ttl = 300

    # If within 20% of expiry, refresh in background
    if remaining_ttl < original_ttl * 0.2:
        if not refresh_in_flight(key):
            async_refresh(key)

    return val
```

Best fit: High-traffic keys with predictable access patterns (product detail pages during flash sales, homepage content). Requires knowing which keys are “hot.”

Trade-off: Refreshes keys that might not be accessed again (wasted work). Requires tracking key heat, which adds complexity.

Comparative Summary

Pattern	Consistency	Write Latency	Read Latency (miss)	Risk
Cache-Aside	Eventual (TTL lag)	Low (async invalidate)	High (DB fetch)	Thundering herd on cold start
Read-Through	Eventual	Low	High (DB fetch)	Cache outage = read failure
Write-Through	Strong	High (sync DB write)	Low	Write amplification
Write-Behind	Eventual (async)	Very Low	Low	Data loss on crash
		n/a	Very Low	

Pattern	Consistency	Write Latency	Read Latency (miss)	Risk
Refresh-Ahead	Strong (proactive)			Wasted refreshes for cold keys

Decision framework: - Default for most services → **Cache-Aside** - Must never serve stale after a write → **Write-Through** - Write throughput is the bottleneck → **Write-Behind** (with durable buffer) - Hot keys, predictable access, zero miss latency → **Refresh-Ahead** - Infrastructure-enforced consistency → **Read-Through**

Eviction Policies

When the cache is full, which entry do you evict?

LRU (Least Recently Used)

- **How:** Doubly-linked list + hash map. On access, move node to head. On eviction, remove from tail.
- **Best for:** General-purpose temporal locality (recently accessed = likely to be accessed again)
- **Weakness:** A single sequential scan (e.g., a batch job) evicts all your hot working set.

LFU (Least Frequently Used)

- **How:** Count accesses per key; evict lowest count.
- **Best for:** Long-lived hot keys that don't need recent access (e.g., popular product pages)
- **Weakness:** New keys get evicted quickly even if they're about to become hot ("cache pollution" by old keys with accumulated counts)

ARC (Adaptive Replacement Cache)

- **How:** Maintains four lists — recently used once (T_1), frequently used (T_2), ghost of recently evicted T_1 (B_1), ghost of recently evicted T_2 (B_2). Adapts balance of T_1/T_2 based on hit patterns.
- **Best for:** Workloads with mixed temporal and frequency patterns
- **Used by:** ZFS, some database buffer pools

Redis default: `allkeys-lru` for cache use cases. Switch to `allkeys-lfu` if your access pattern is frequency-driven.

Cache Vulnerability Vectors

1. Cache Penetration

What happens: Requests for keys that don't exist in cache *or* DB bypass the cache every time, hitting the DB on every request.

```
Attacker: GET /user/99999999 (doesn't exist)
→ Cache miss
→ DB query (returns null)
→ Cache does NOT cache null (common mistake)
→ Next request: same path, same DB hit
→ 10K RPS → 10K DB queries for non-existent users
```

Fix 1 — Null caching: Cache the null result with a short TTL.

```
user = db.get_user(user_id)
if user is None:
    redis.setex(f"user:{user_id}", "NULL", ttl=60)    # Short TTL
    return None
redis.setex(f"user:{user_id}", serialize(user), ttl=3600)
```

Fix 2 — Bloom filter (better for attack scale):

```
# On startup, populate bloom filter with all valid user IDs
bloom = BloomFilter(capacity=10_000_000, error_rate=0.01)
for user_id in db.get_all_user_ids():
    bloom.add(user_id)

def get_user(user_id: int):
    if user_id not in bloom:
        return None    # Definitely doesn't exist — skip DB entirely
    return cache_or_db_lookup(user_id)
```

When to use Bloom filter: User ID space is large and unpredictable; attacker is enumerating IDs. Null caching works for accidental misses; Bloom filters work for adversarial ones.

2. Cache Avalanche

What happens: A large portion of cached keys expire simultaneously, causing a wave of DB requests.

Scenario:

03:00 AM: Daily cache warm-up sets 500K keys with TTL=86400s
03:00 AM next day: All 500K keys expire at the same second
→ 500K concurrent DB queries → DB overwhelmed

Fix — TTL jitter: Add randomness to expiration times.

```
import random

BASE_TTL = 86400 # 24 hours

def set_with_jitter(key: str, value, base_ttl: int = BASE_TTL):
    # ±10% jitter spreads expiry over a 2-hour window
    jitter = random.randint(-base_ttl // 10, base_ttl // 10)
    actual_ttl = base_ttl + jitter
    redis.setex(key, value, actual_ttl)
```

Fix — Layered TTLs: Different content types have different base TTLs, so they don't all expire together even without jitter.

```
User profiles:    TTL = 3600s    (1 hr)
Product pages:   TTL = 300s     (5 min)
Inventory counts: TTL = 30s      (30 sec)
Static config:   TTL = 86400s   (24 hr)
```

3. Cache Stampede (Thundering Herd)

What happens: A single popular key expires. Before any request can re-populate it, many concurrent requests all get a cache miss and all fire DB queries simultaneously.

```
10K concurrent users → product page for iPhone
Cache key expires at T=0
T=0: All 10K get cache miss
T=0: All 10K query DB
T=0.1: DB has 10K concurrent queries → OOM / timeout
```

Fix 1 — Probabilistic Early Expiration (PER):

```
import math, random, time

def get_with_per(key: str, db_fetch_fn, ttl: int, beta: float = 1.0):
    """
    Beta controls aggressiveness:
    beta=1.0: standard PER
```

```

    beta > 1: refresh earlier (more eager)
    """
    cached, expire_time = redis.get_with_expiry(key)

    if cached:
        # Compute probabilistic early expiry
        remaining = expire_time - time.time()
        gap = -math.log(random.random()) * beta    # Exponential random

        if gap > remaining:
            # Proactively refresh – only ONE request does this each time
            value = db_fetch_fn()
            redis.setex(key, serialize(value), ttl)
            return value
        return deserialize(cached)

    # True miss – fetch from DB
    value = db_fetch_fn()
    redis.setex(key, serialize(value), ttl)
    return value

```

Fix 2 — Single-Flight (Request Coalescing):

```

import asyncio

_inflight: dict[str, asyncio.Future] = {}

async def get_with_single_flight(key: str, db_fetch_fn) -> any:
    cached = redis.get(key)
    if cached:
        return deserialize(cached)

    if key in _inflight:
        # Already in progress – wait for it
        return await _inflight[key]

    fut = asyncio.get_event_loop().create_future()
    _inflight[key] = fut

    try:
        value = await db_fetch_fn()
        redis.setex(key, serialize(value), 300)
        fut.set_result(value)
        return value
    except Exception as e:
        fut.set_exception(e)
        raise
    finally:
        _inflight.pop(key, None)

```

Fix 3 — Mutex (Redis distributed lock):

```
def get_with_lock(key: str, db_fetch_fn) -> any:
    cached = redis.get(key)
    if cached:
        return deserialize(cached)

    lock_key = f"{key}:lock"
    lock_acquired = redis.set(lock_key, "1", nx=True, ex=10)    # 10s lock TTL

    if lock_acquired:
        try:
            value = db_fetch_fn()
            redis.setex(key, serialize(value), 300)
            return value
        finally:
            redis.delete(lock_key)
    else:
        # Another process holds lock – wait and retry
        time.sleep(0.1)
        return redis.get(key) or get_with_lock(key, db_fetch_fn)    # retry
```

Which stampede fix to use:

Fix	Complexity	Best for
PER	Low	High-traffic keys; can tolerate occasional stale
Single-Flight	Medium	Async services; Python/Go async runtimes
Mutex (Redis lock)	Medium	Synchronous services; strict freshness needed
Pre-warming	High	Predictable hot events (sales, launches)

Interview Question 1 — Flash Sale Stampede

Prompt: Your cache experiences stampede during flash sales. How do you prevent database overload when 10K users simultaneously request the same product page?

Multi-Layer Defense

```
Layer 1 – Pre-warming (before the event):
    30 min before sale: Batch-load predicted hot product pages
    extended_ttl = sale_duration + 30 min    (ensure cache outlasts the sale)
```

Layer 2 – TTL jitter (prevent synchronized expiry):

`TTL = base_ttl ± random(0, 60s)`

Layer 3 – Single-flight (prevent thundering herd on miss):

At most 1 DB query per key at any time

All other requesters wait for the in-flight result

Layer 4 – Circuit breaker (prevent DB cascade):

If DB error rate > 20% over 30s → open circuit

Serve stale cache beyond TTL (graceful degradation)

Alert on-call

Concrete latency budget:

Normal path (cache hit): < 2ms Redis read

Stampede path (single-flight): First requester: DB query ~20ms

Other 9,999: wait ~20ms, then serve cached result

Net DB load: 1 query instead of 10,000

Interview Question 2 — Rate Limiter with Cache Avalanche Risk

Prompt: You're building a rate limiter for an API. It stores per-IP request counts in Redis with TTL=60s. How do you prevent avalanche when many IPs were created at the same time (e.g., batch onboarding)?

Problem

If 100K users all start their first request within a 1-second window, their rate limit keys are created with the same expiration time. At T+60s, 100K keys expire simultaneously → 100K DB checks (or Redis resets) at once.

Solution

```
import random, time

def record_request(ip: str, rate_limit: int = 1000):
    key = f"ratelimit:{ip}"
    current = redis.get(key)

    if current is None:
        # New key — add per-IP jitter based on IP hash
        # Same IP always gets same jitter → consistent window
        ip_jitter = hash(ip) % 10    # 0-9 seconds
        ttl = 60 + ip_jitter
```

```

    redis.set(key, 1, ex=ttl)
    return True

count = int(current)
if count >= rate_limit:
    return False    # Rate limited

redis.incr(key)
return True

```

Why hash-based jitter: Pure random jitter means the same IP might get a different window each minute. Hash-based jitter gives consistent window lengths per IP while still distributing expiry times globally.

Interview Question 3 — Write-Behind for High-Throughput Counters

Prompt: Your like/view counter service needs to handle 500K increments/second on popular posts. Direct DB writes are impossible at that rate. Design the write path.

Solution: Write-Behind with Durable Buffer

```

Write path:
  Increment request → Redis INCR (in-memory, atomic, ~0.1ms)
  → Background flusher polls Redis every 500ms
  → Batch-writes accumulated counts to DB

Read path:
  Read from Redis (always fresh, fast)

Durability:
  Redis AOF persistence (fsync every second)
  → At most 1 second of count loss on Redis crash

Recovery:
  On restart, Redis loads from AOF
  Flusher resumes from last successful DB write

```

```

import time, threading
from collections import defaultdict

pending_increments = defaultdict(int)
lock = threading.Lock()

def increment_view(post_id: str):
    # Atomic Redis increment (fast path)
    new_count = redis.incr(f"views:{post_id}")

```

```

    # Track for batch flush
    with lock:
        pending_increments[post_id] += 1

    return new_count

def flush_to_db():
    while True:
        time.sleep(0.5)    # Flush every 500ms

        with lock:
            batch = dict(pending_increments)
            pending_increments.clear()

        if batch:
            # Single DB round-trip for all pending increments
            db.bulk_increment_views(batch)
            # e.g.: UPDATE posts SET views = views + N WHERE id = ?

# Run flusher in background thread
threading.Thread(target=flush_to_db, daemon=True).start()

```

Throughput math:

```

500K increments/sec → Redis INCR: ~0.01ms each = feasible
Batch size at 500ms interval:  $500K \times 0.5 = 250K$  increments
DB batch write: 1 multi-row UPDATE every 500ms
DB load: 2 queries/sec instead of 500K/sec

```